Contents:

- Openings
- Information and procedures
- Course descriptions
- Background

Operating Systems

This is a course for 3th year students who are studying major or minor degree in computer science.

Dr. Hatem Moharram

Books:

Operating Systems: Internals and Design Principles by Wialliam Stallings.

Modern Operating Systems by Andrew S. Tanenbaum.

What do you need to study OS?

- Being an efficient programmer using C++ or Java.
- Have a good idea of Computer architecture and/or Computer Organization.
- Have a good idea in Windows or Unix.

The course consists of the following parts:

- Part 1: Background

Provides an overview of computer architecture and organization, with emphasis on topics that relate to operating system design.

- Part 2: Processes

  presents a detailed analysis of processes, multithreading, symmetric multiprocessing(SMP), and microkernales.

- Part 3 Memory

  provides a comprehensive survey of techniques for memory management, including virtual memory.

- Part 3 Scheduling

provides a comprehensive discussion of various approaches to process scheduling.

- Part 5 Input/Output and Files

Examines the issues involved in OS control of the I/O function and an overview of the file management.

- Part 6 Distributed Systems and Security

Examines the major trends in the networking computer systems, including TCP/IP, client/server computing, and clusters.

Computer System Overview

Chapter 1

By: Dr. Hatem Moharram

Operating System

- **Exploits the hardware resources of one or more processors to provides a set of services to system users**
- **The OS manages secondary memory and I/O devices on behalf of its users.**

it is important to have some understanding of the underlying computer system hardware before we begin our examination of operating systems.

1.1 BASIC ELEMENTS

- **a computer consists of:**
  - **processor,**
  - **memory, and**
  - **I/O components, with one or more modules of each type.**

- **These components are interconnected in some fashion to achieve the main function of the computer, which is to execute programs.**

Basic Elements

there are four main structural elements:

1- Processor:

Controls the operation of the computer and performs its data processing functions.

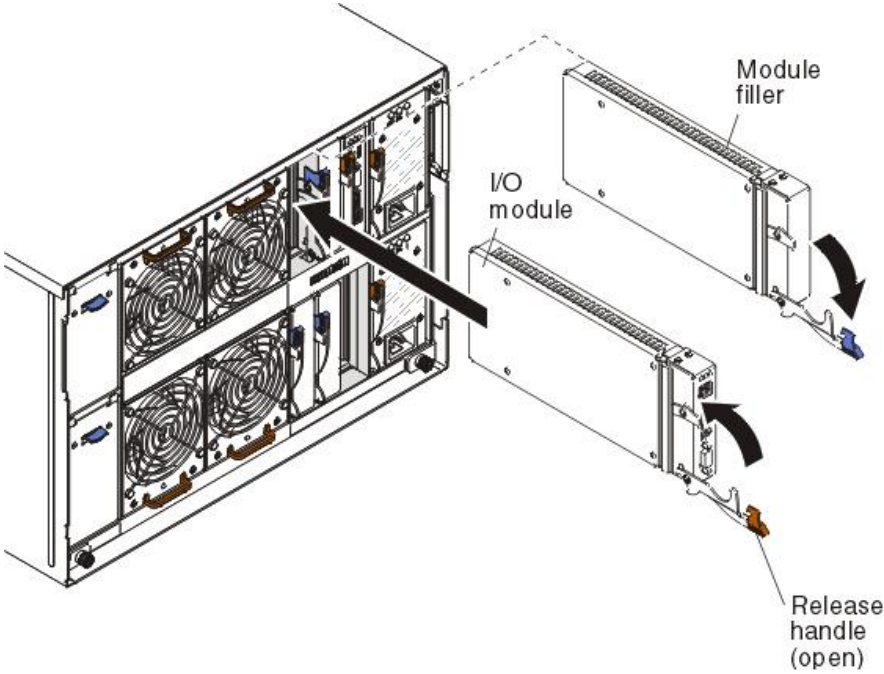
When there is only one processor, it is often referred to as the central processing unit (CPU).

2- Main Memory

- Stores data and programs.
- referred to as real memory or primary memory
- volatile

3- I/O modules

– **Move data between the computer and its external environment.**

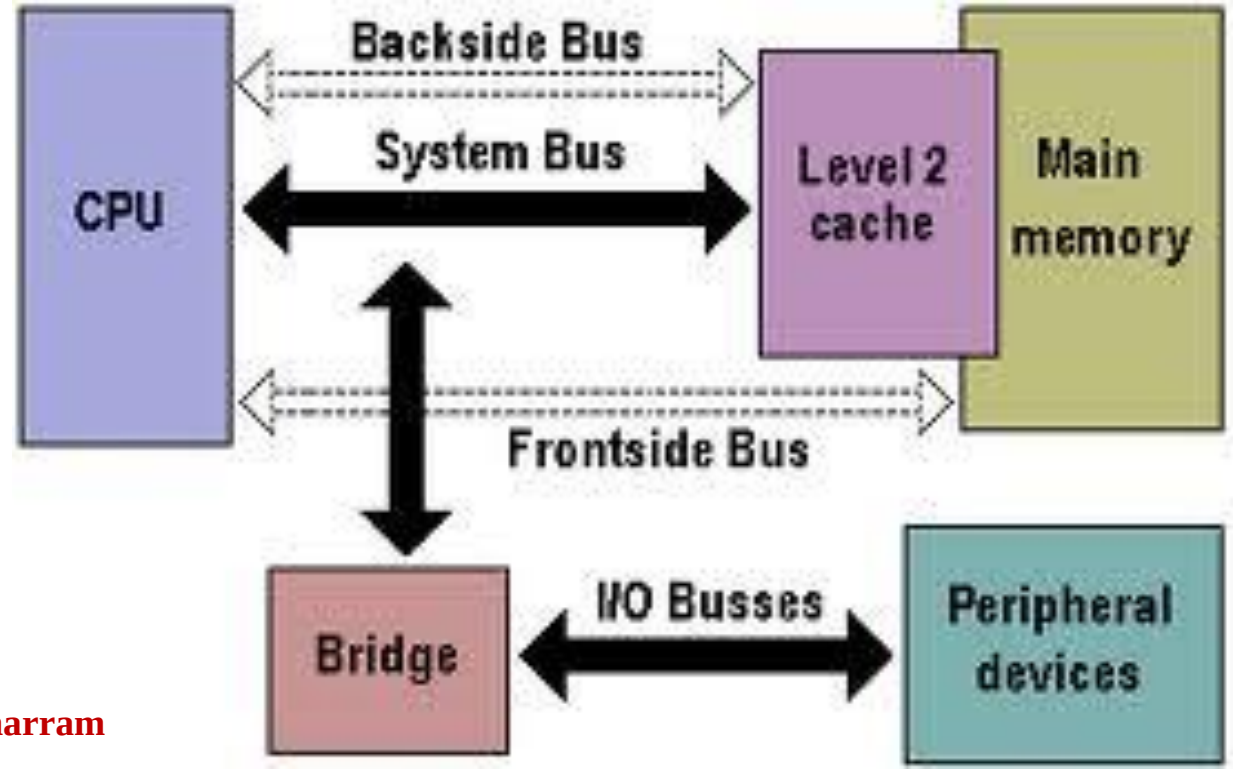


external environment:

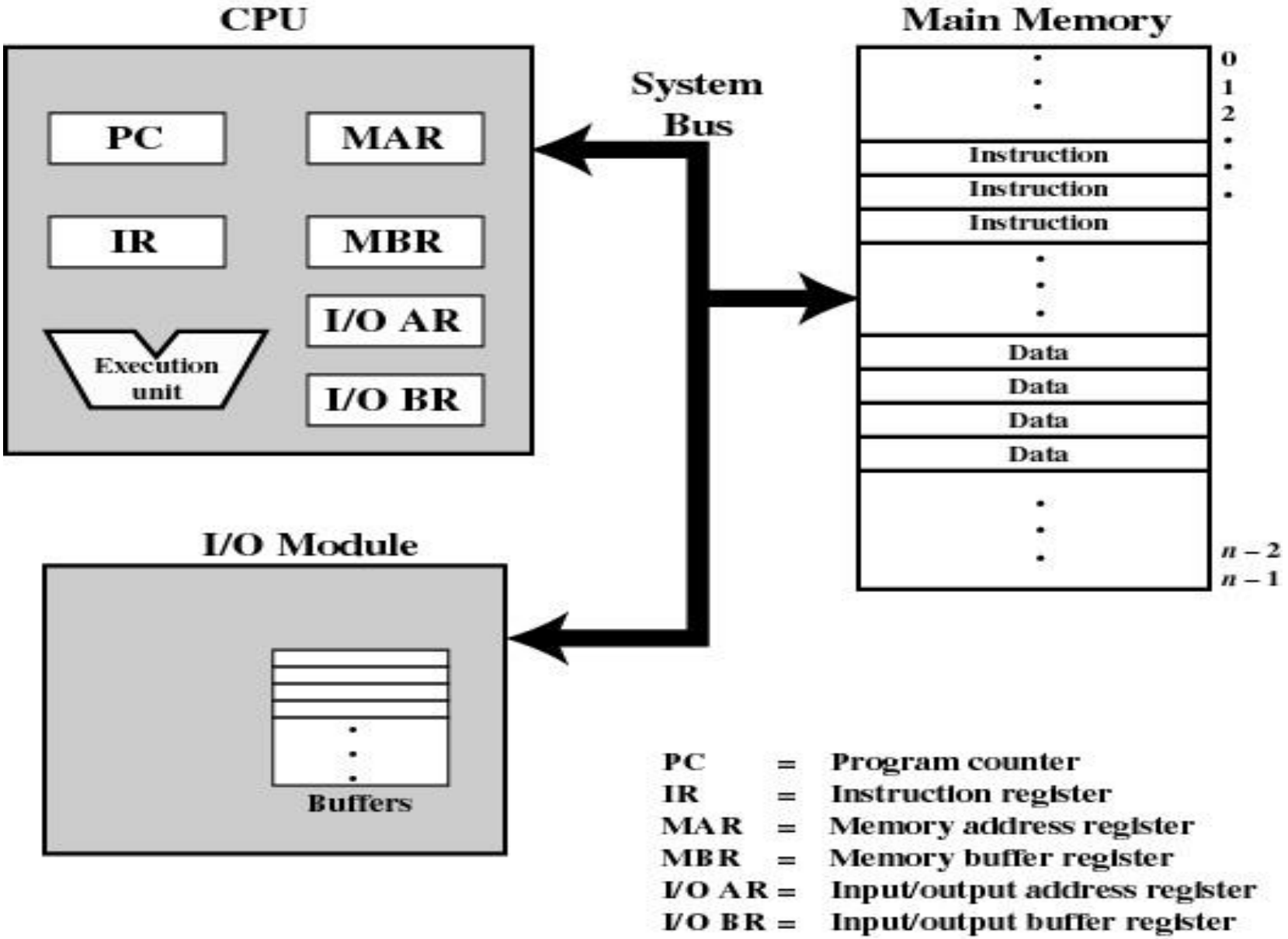
- **secondary memory devices (e. g., disks),**
- **communications equipment**
- **terminals**

4- System bus

Provides for communication among processors, main memory, and I/O modules.



Top-Level Components



Dr. Hatem Moharram

Figure 1.1 Computer Components: Top-Level View

to exchange data with memory the processor uses two internal registers:

1- memory address register (MAR):

contains the address in memory for the next read or write

2- memory buffer register (MBR):

contains the data to be written into memory or which receives the data read from memory.

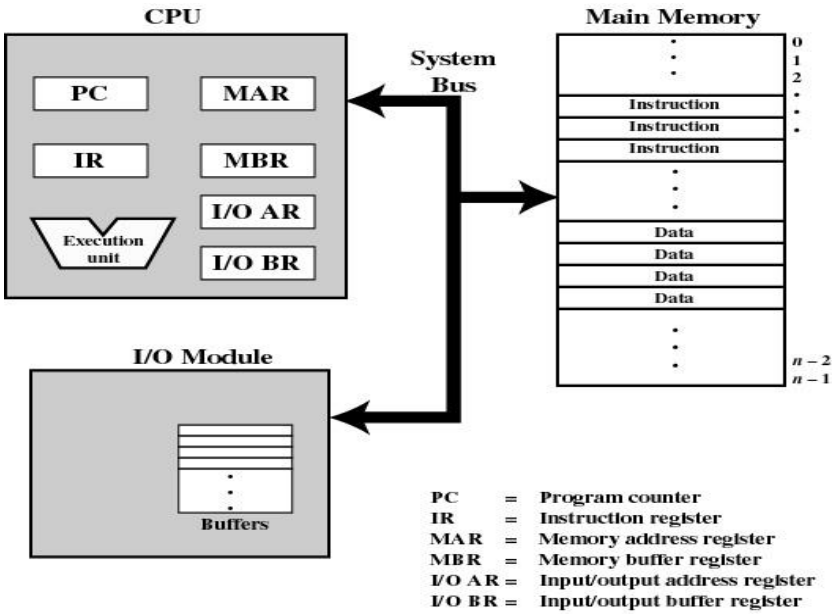


Figure 1.1 Computer Components: Top-Level View

to exchange data with I/O devices the processor uses two internal registers:

1- I/O address register (I/OAR):

specifies a particular I/O device.

2- I/O buffer register (I/OBR):

is used for the exchange of data between an I/O module and the processor.

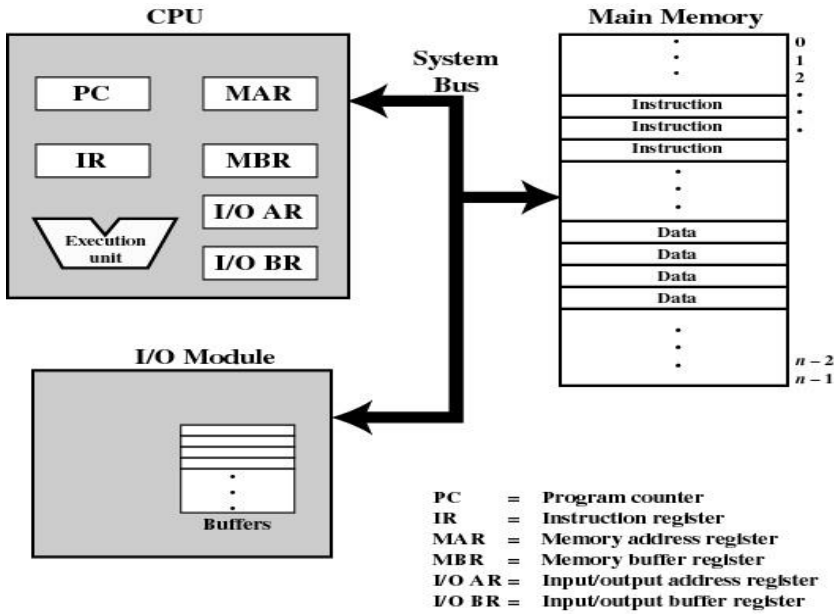


Figure 1.1 Computer Components: Top-Level View

memory module

A memory module consists of a set of locations, defined by sequentially numbered addresses.

Each location contains a bit pattern that can be interpreted as either an instruction or data.

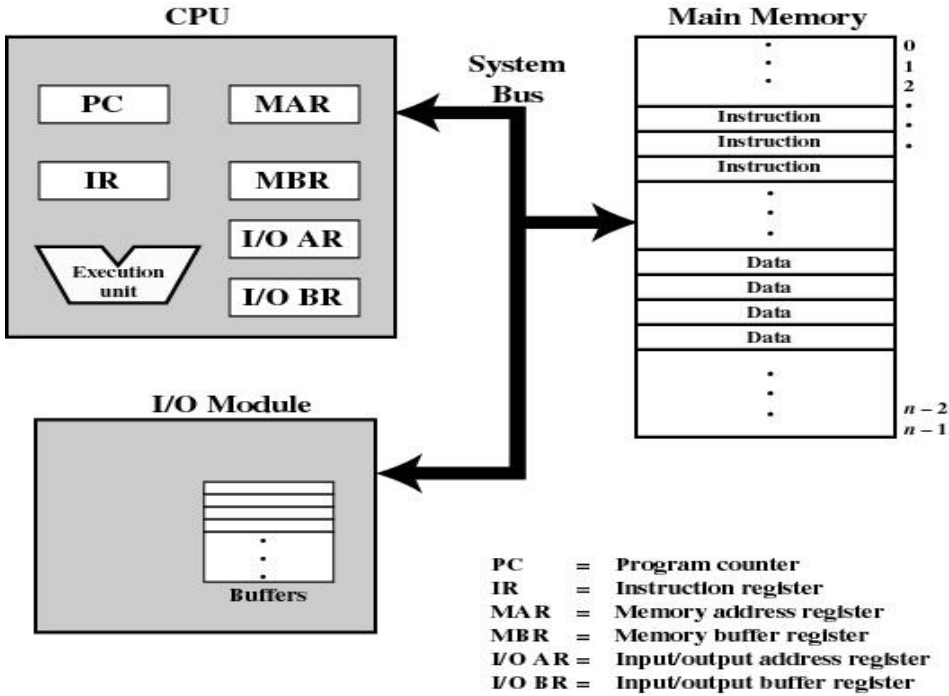


Figure 1.1 Computer Components: Top-Level View

I/O modules

An I/O module transfers data from external devices to processor and memory, and vice versa.

It contains internal buffers for temporarily holding data until they can be sent on.

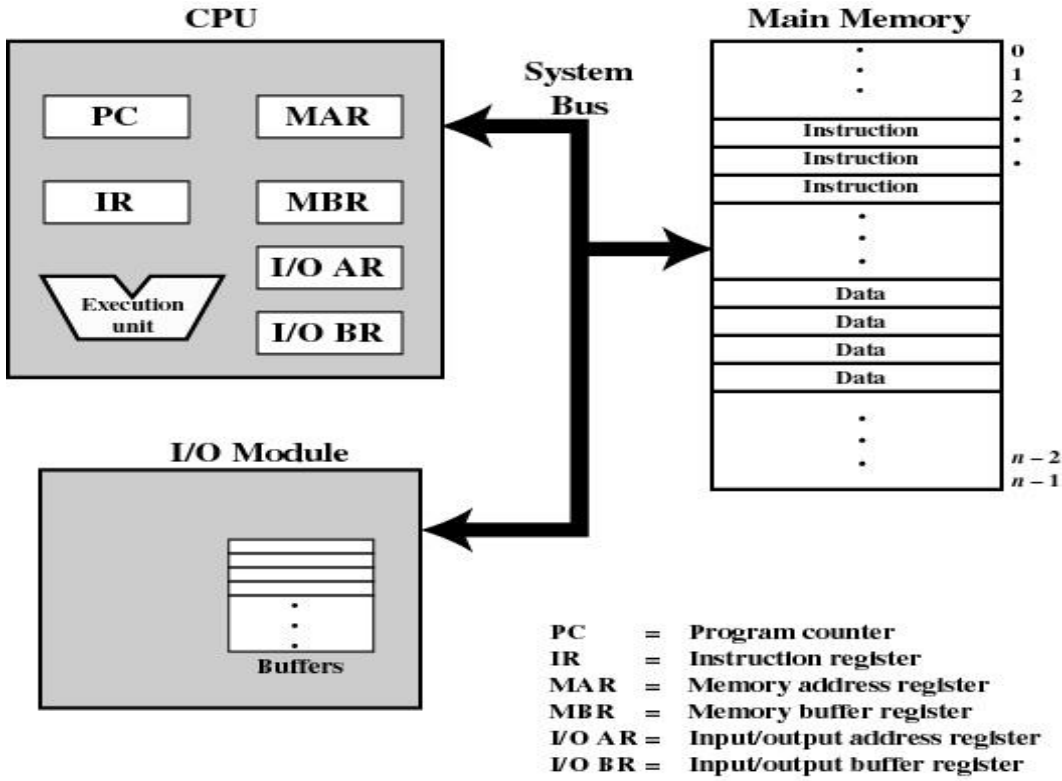


Figure 1.1 Computer Components: Top-Level View

1.2 PROCESSOR REGISTERS

A processor includes a set of registers that provide memory that is faster and smaller than main memory.

Processor registers serve two functions:

1- User-visible registers

Enable assembly language programmer to minimize main-memory references by optimizing register use

2- Control and status registers

- **Used by processor to control operating of the processor**
- **Used by OS routines to control the execution of programs**

User-Visible Registers

- **May be referenced by machine language that the processor executes.**
- **Available to all programs - application programs and system programs**

Types of registers

- Data Registers: **can be assigned to a variety of functions by the programmer.**

- Address Registers: **contain main memory addresses of data and instructions, or they contain a portion of the address that is used in the calculation of the complete or effective address.**

User-Visible Registers

Data Registers

- **can be assigned to a variety of functions by the programmer.**
- **they are general purpose in nature and can be used with any machine instruction that performs operations on data.**
- **there may be dedicated registers for floating-point operations and others for integer operations.**

User-Visible Registers

Address Registers

contain main memory addresses of data and instructions,

or

they contain a portion of the address that is used in the calculation of the complete or effective address.

Control and Status Registers

employed to control the operation of the processor. On most processors, most of these are not visible to the user.

1- Program Counter (PC)

2- Instruction Register (IR)

3-Program Status Word (PSW)

Control and Status Registers

1- Program Counter (PC)

- Contains the address of an instruction to be fetched

2- Instruction Register (IR)

- Contains the instruction most recently fetched

Control and Status Registers

3- Program Status Word (PSW)

contains status information

- condition codes
- Interrupt enable/disable
- Supervisor/user mode

sign	interrupt	mode	condition	……	……	……..	overfllow

Control and Status Registers

3- Program Status Word (PSW)

- **Condition Codes (also referred to as *flags)***
  - **Bits set by the processor hardware as a result of operations**
  - **Can be accessed by a program but not altered**
  - **Examples**
    - **positive result**
    - **negative result**
    - **zero**
    - **Overflow**

1.3 INSTRUCTION EXECUTION

instruction processing consists of two steps:

1- The processor reads (*fetches*) *instructions from memory one at a time*

***2- executes each* instruction.**

- **Program execution consists of repeating the process of instruction fetch and instruction execution.**

- **Instruction execution may involve several operations and depends on the nature of the instruction.**

The processing required for a single instruction is called an *instruction cycle.*

Instruction Cycle

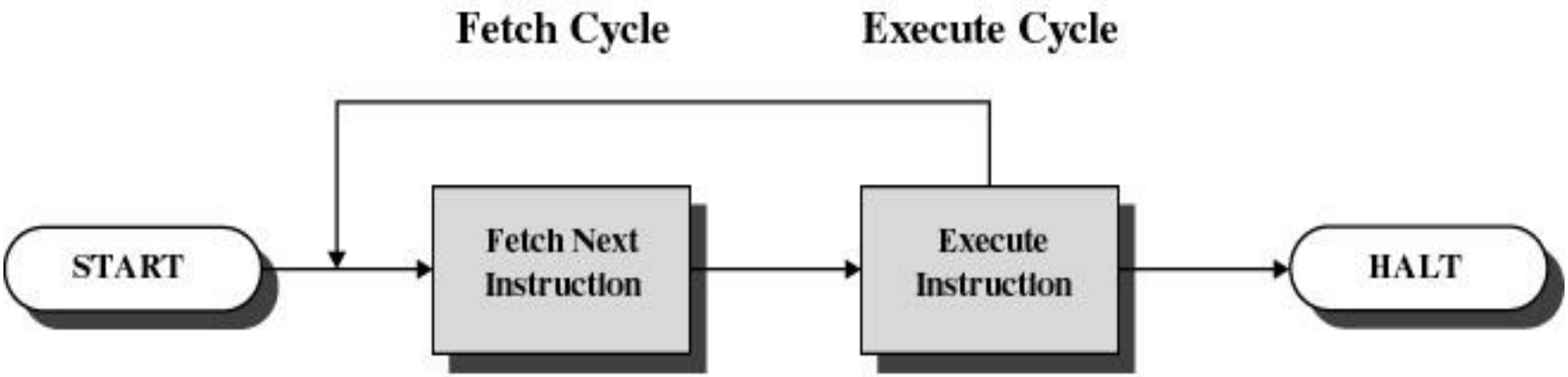


Figure 1.2 Basic Instruction Cycle

Instruction Fetch and Execute

At the beginning of each instruction cycle:

1- The processor fetches the instruction from memory

(Program counter (PC) holds address of the instruction to be fetched next)

2- Program counter is incremented after each fetch

(Unless instructed otherwise)

3-The fetched instruction is loaded into the instruction register (IR).

4- The processor interprets the instruction and performs the required action.

Instruction Fetch and Execute

Types of instructions

1- Processor-memory

transfer data between processor and memory

2- Processor-I/O

data transferred to or from a peripheral device

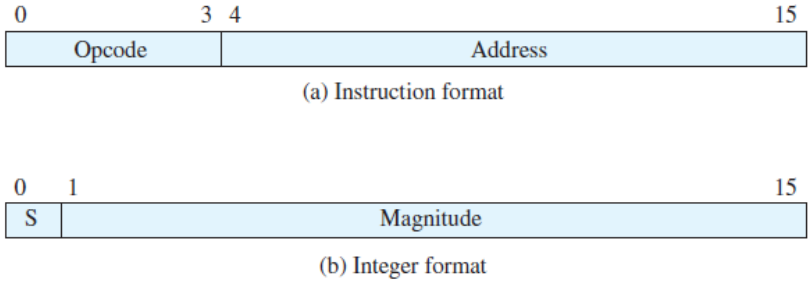
3- Data processing

arithmetic or logic operation on data

4- Control

alter sequence of execution

Example of Program Execution



(a) Instruction format

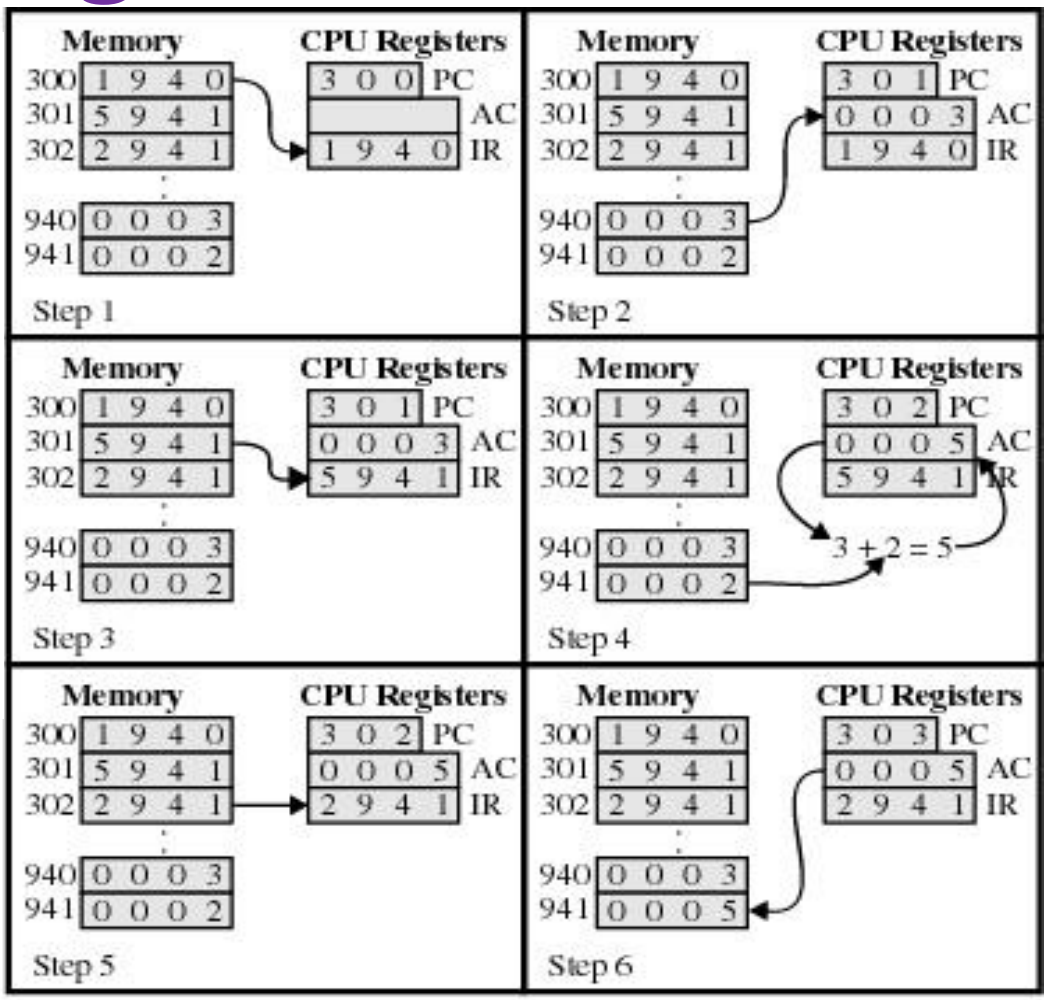
(b) Integer format

Program counter (PC) = Address of instruction
Instruction register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from memory
0010 = Store AC to memory
0101 = Add to AC from memory

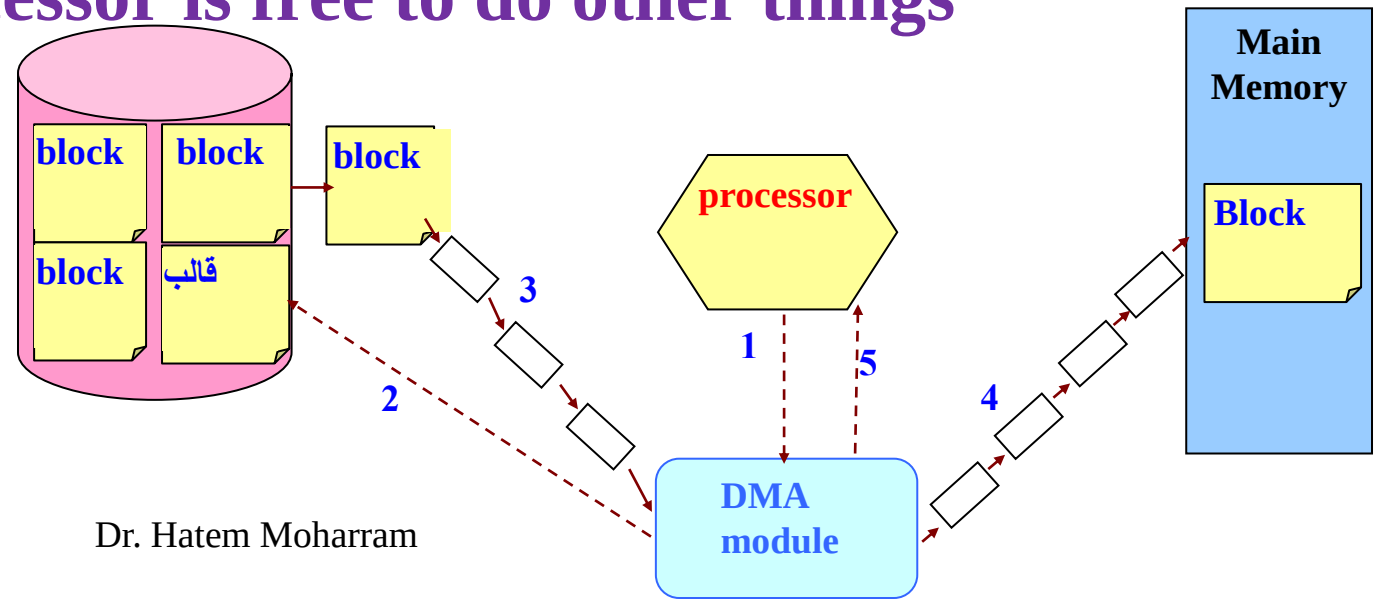
(d) Partial list of opcodes



adds the contents of the memory word at address 940 to the contents of the memory word at address 941 and stores the result in the latter location.

Direct Memory Access (DMA)

- I/O exchanges occur directly with memory
- Processor grants I/O module authority to read from or write to memory
- Relieves the processor responsibility for the exchange
- Processor is free to do other things



Interrupts

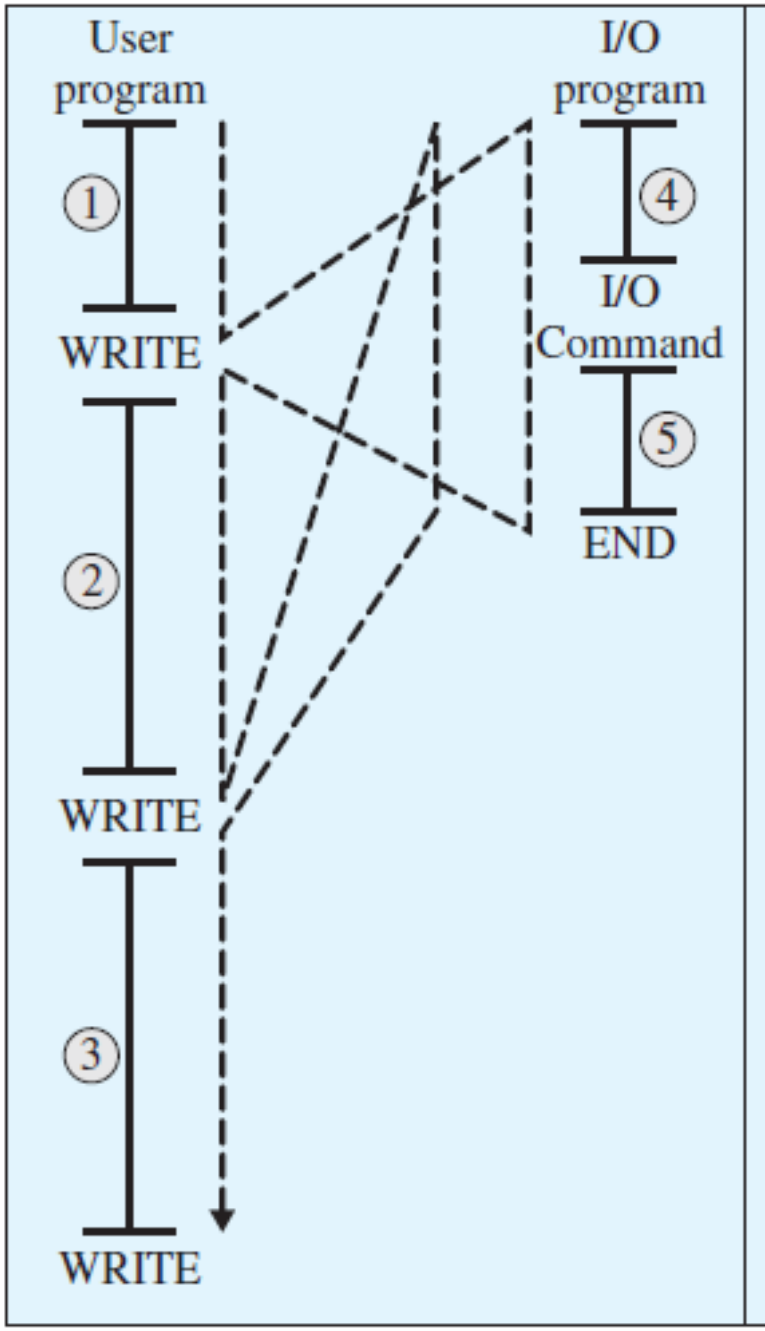
- **all computers provide a mechanism by which other modules (I/O, memory) may interrupt the normal sequencing of the processor.**
- **a way to improve processor utilization.**
- **An interruption of the normal sequence of execution**

The I/O program consists of three sections:

1- A sequence of instructions(4), to prepare for the actual I/O operation. This may include copying the data to be output into a special buffer and preparing the parameters for a device command.

2- The actual I/O command, once this command is issued, the program must wait for the I/O device to perform the requested function repeatedly performing a test operation to determine if the I/O operation is done.

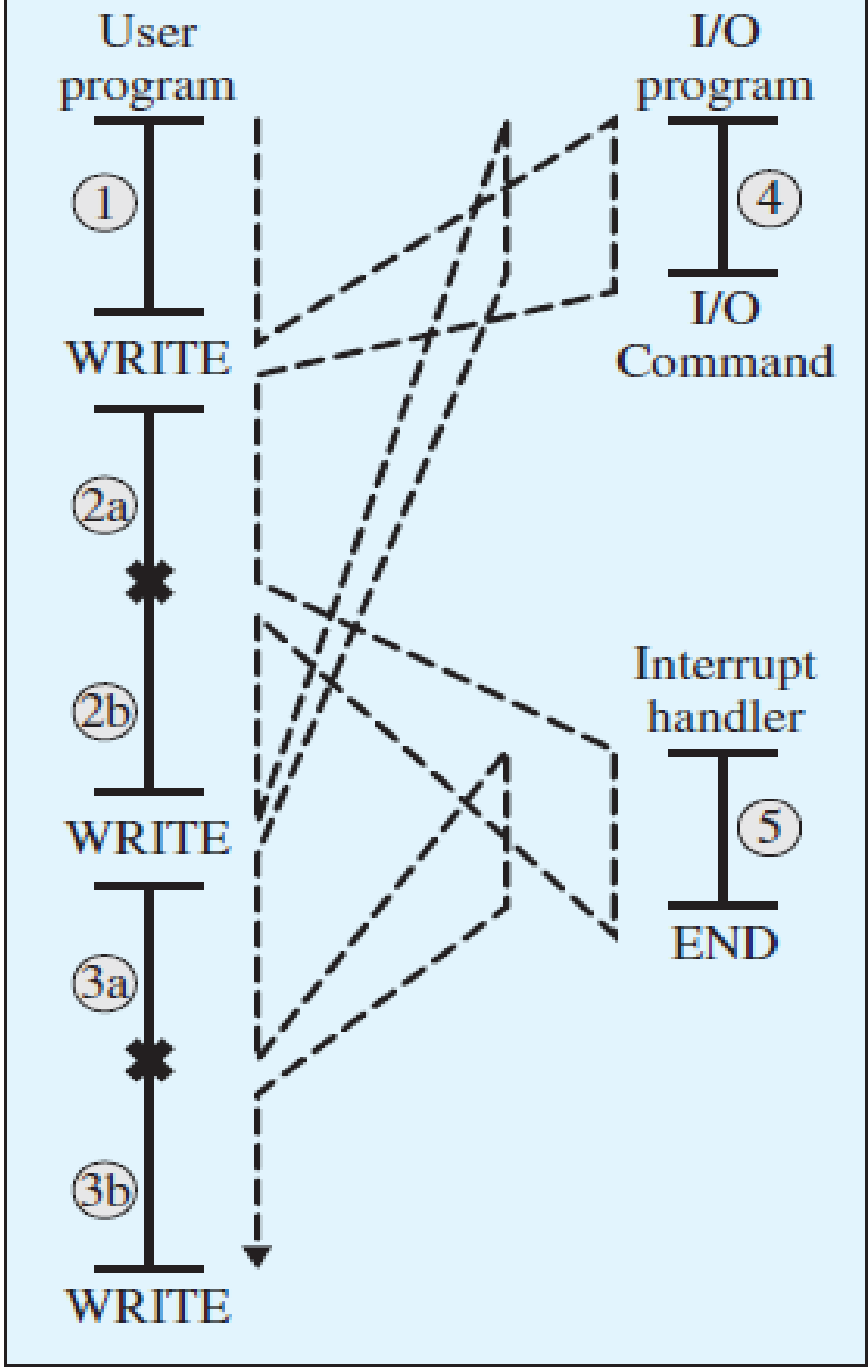
3- A sequence of instructions(5), to complete the operation. This may include setting a flag indicating the success or failure of the operation.



(a) No interrupts

With interrupts,

the processor can be engaged in executing other instructions while an I/O operation is in progress.



(b) Interrupts; short I/O wait

Classes of Interrupts

- **Program**

  Generated by some condition that occurs as a result of an instruction execution.

  - **arithmetic overflow**
  - **division by zero**
  - **execute illegal instruction**
  - **reference outside user's memory space**

- **Timer**

Generated by a timer within the processor.

Allows the operating system to perform certain functions on a regular basis.

- **I/O**

Generated by an I/O controller

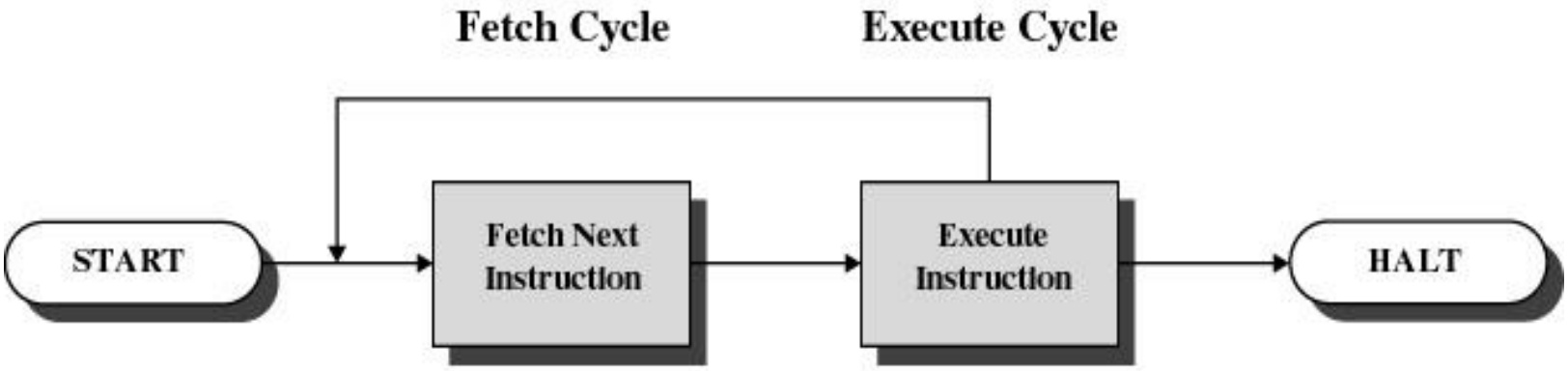
To signal normal completion of an operation

To signal a variety of error conditions

- **Hardware failure**

Generate by a failure, such as power failure or memory error

Without interrupts,



- After each write operation, the processor must pause and remain idle until the printer catches up.

- This is a very wasteful use of the processor

- The write calls are to an I/O routine that is a system utility and that will perform the actual I/O operation.

Interrupts and the Instruction Cycle

- **When a Write call occurs**

- **the processor executes the I/O program witch consists only of the preparation code and the actual I/O command (few instructions).**

- **Control returns to the user program.**

- **Meanwhile, The external device is busy accepting data from the computer memory and printing it.**

- **The processor executes the instructions in the user program.**

- **When the external device becomes ready, the I/O module sends an interrupt request signal to the processor.**

- **The processor suspends the operation of the current program.**

- **The processor branch off to a routine to service that particular I/O device, the interrupt handler.**

- **The processor resumes the original execution after the device is serviced.**

-Notes

1- for the user program, an interrupt suspends the normal sequence of execution.

2- When the interrupt processing is completed, execution resumes.

3- the processor and the OS are responsible for suspending and resuming the user program.

Interrupt Cycle

Interrupt stage:

In the interrupt stage the processor checks to see if any interrupts have occurred (the presence of an interrupt signal).

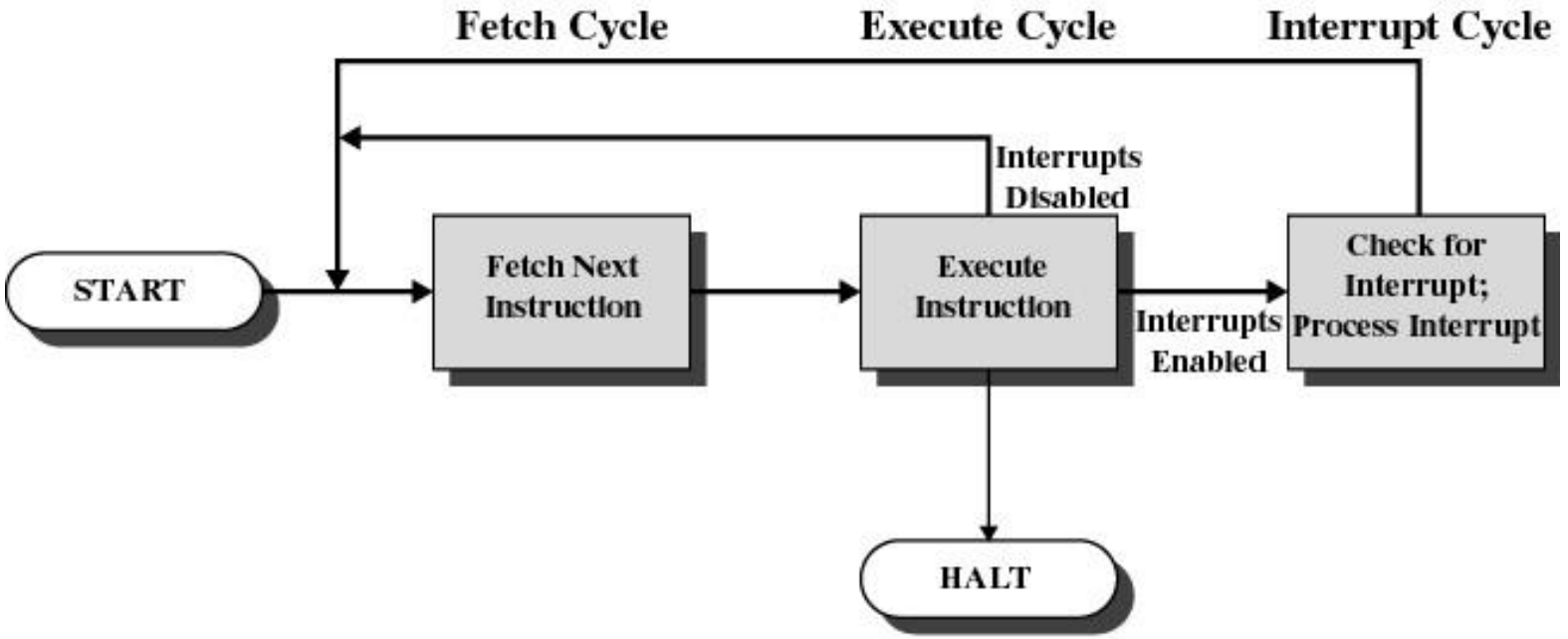


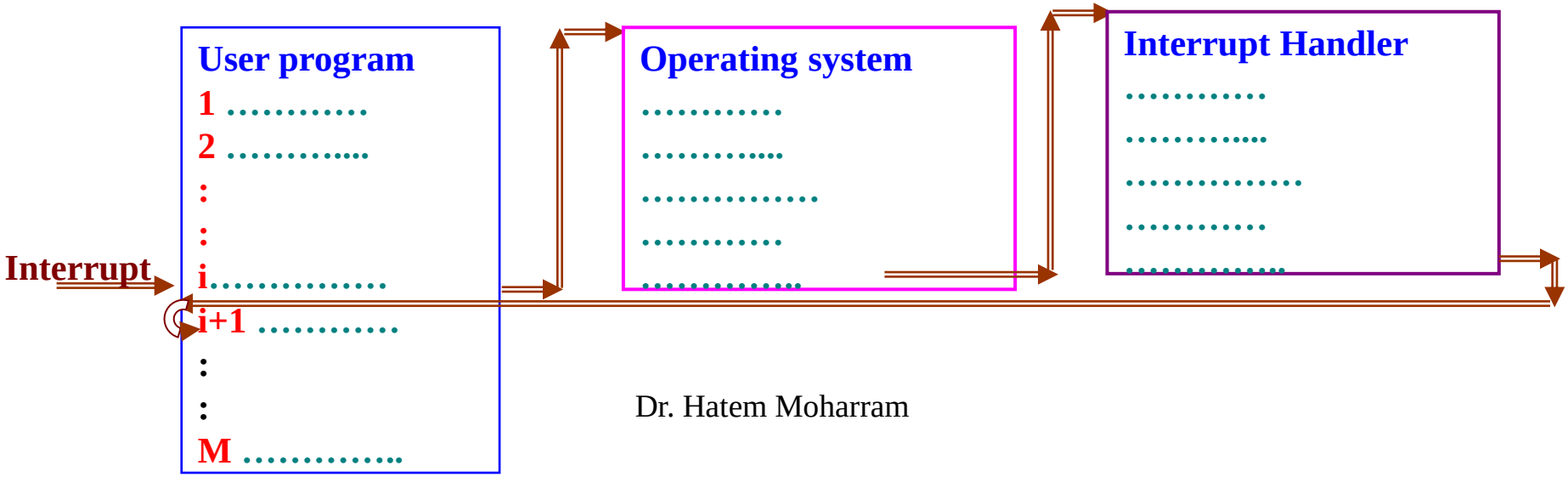
Figure 1.7 Instruction Cycle with Interrupts

Interrupt Handler

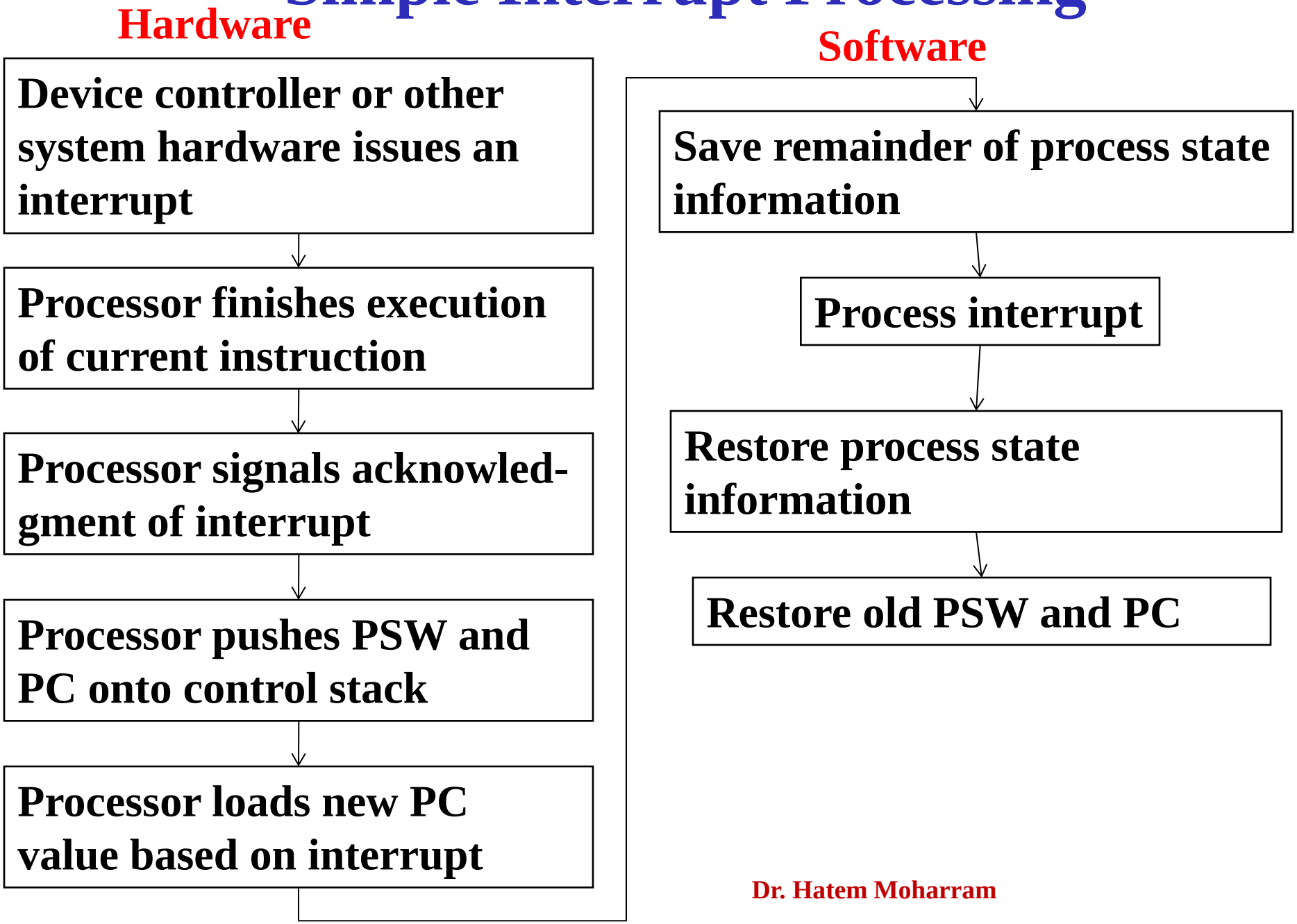
- **A program that determines nature of the interrupt and performs whatever actions are needed**
- **Control is transferred to this program**
- **Generally part of the operating system**

Interrupt Cycle

- **Processor checks for interrupts**
- **If no interrupts fetch the next instruction for the current program**
- **If an interrupt is pending, suspend execution of the current program, and execute the interrupt handler**



Simple Interrupt Processing

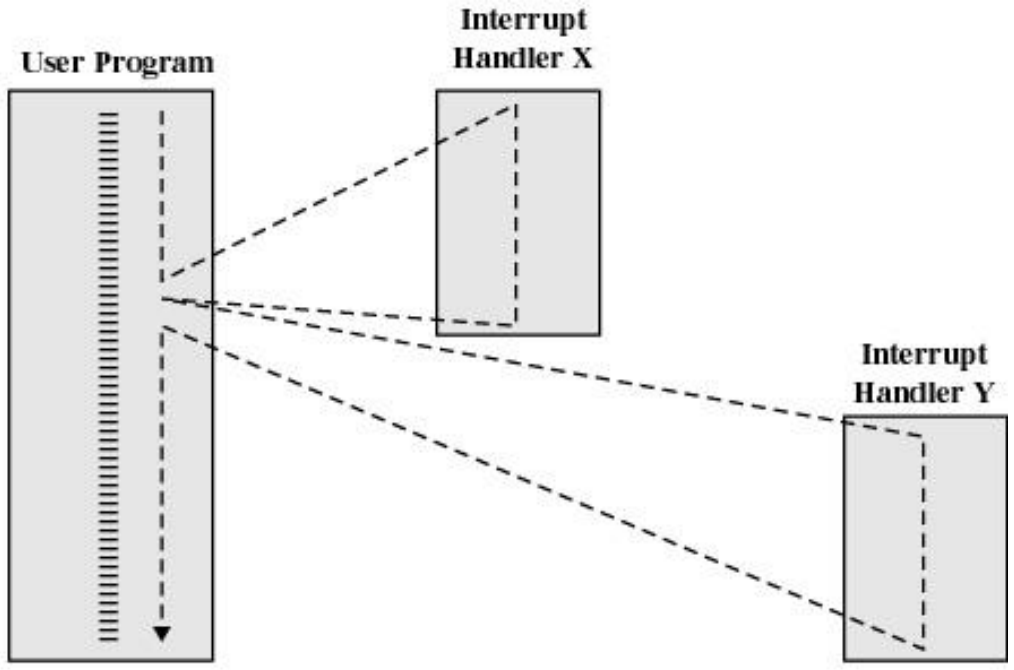


Multiple Interrupts

Two approaches:

1- Disable interrupts while an interrupt is being processed (Sequential Order)

- **Processor ignores any new interrupt request signals**



(a) Sequential Interrupt processing

Multiple Interrupts
Sequential Order

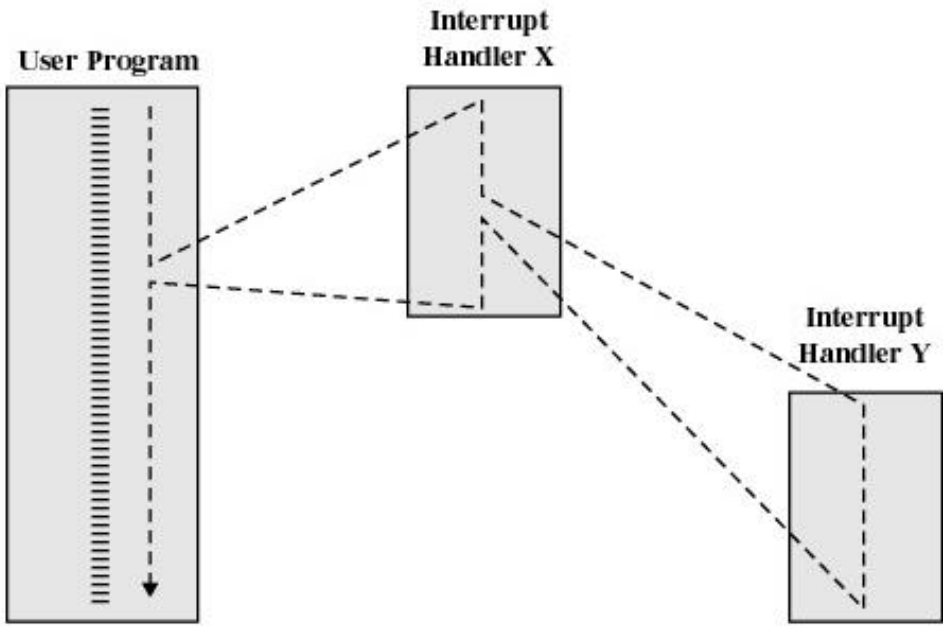
- Disable interrupts so processor can complete task
- Interrupts remain pending until the processor enables interrupts
- After interrupt handler routine completes, the processor checks for additional interrupts
- Does not take into consideration relative priority or time-critical needs.

2- Multiple Interrupts (Priorities)

Higher priority interrupts cause lower-priority interrupts to wait

Causes a lower-priority interrupt handler to be interrupted

- **Ex.: when input arrives from communication line, it needs to be absorbed quickly to make room for more input**



(b) Nested interrupt processing

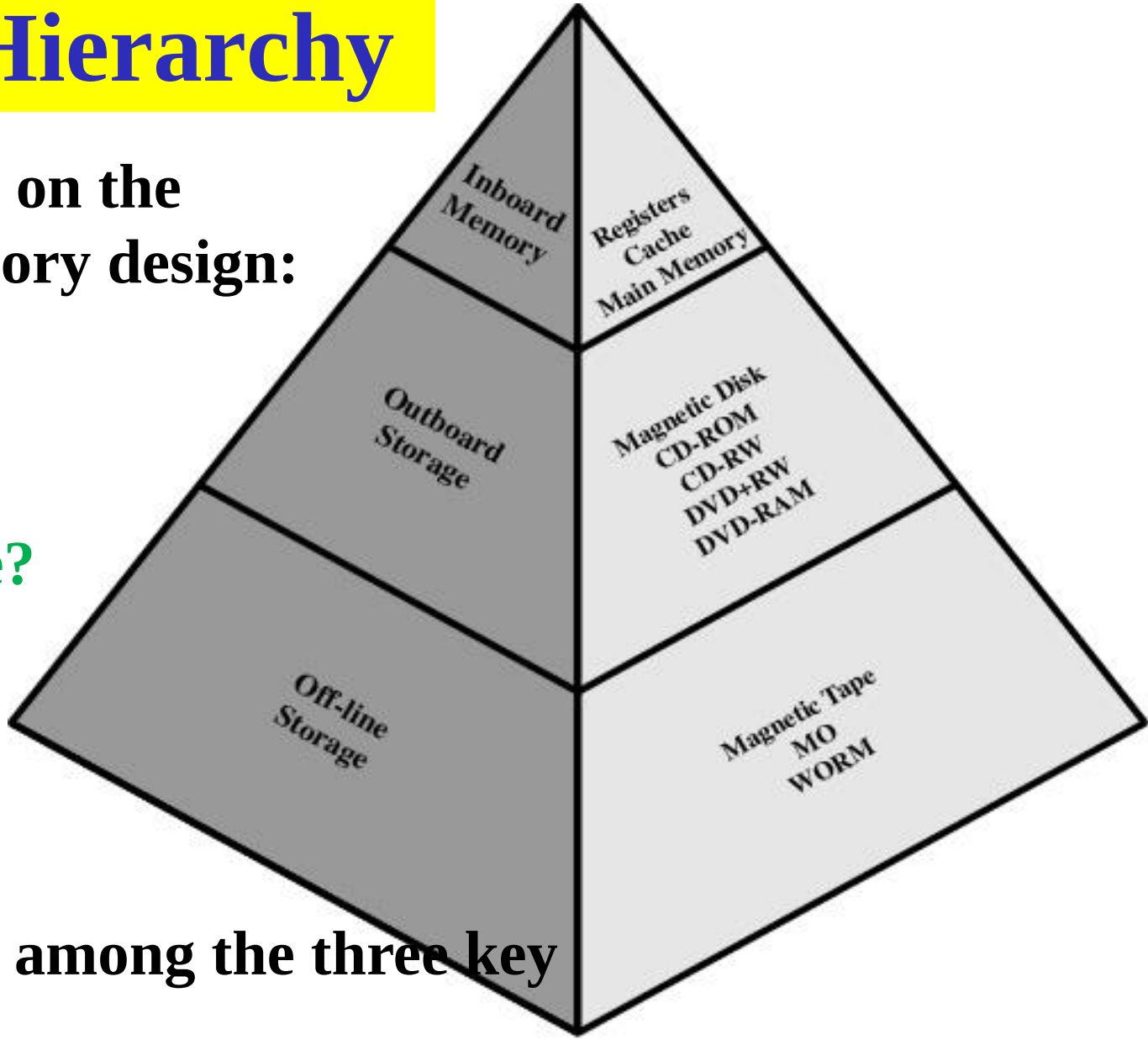
Multiprogramming

- **Even with the use of interrupts, a processor may not be used very efficiently.**
- **Processor has more than one program to execute**
- **The sequence the programs are executed depend on their relative priority and whether they are waiting for I/O**
- **After an interrupt handler completes, control may not return to the program that was executing at the time of the interrupt**

Memory Hierarchy

Three constrains on the computer's memory design:

1- how much?
2- how fast?
3- how expensive?



There is tradeoff among the three key characteristics:

Cost capacity access time.

Figure 1.14 The Memory Hierarchy

Faster access time, greater cost per bit.

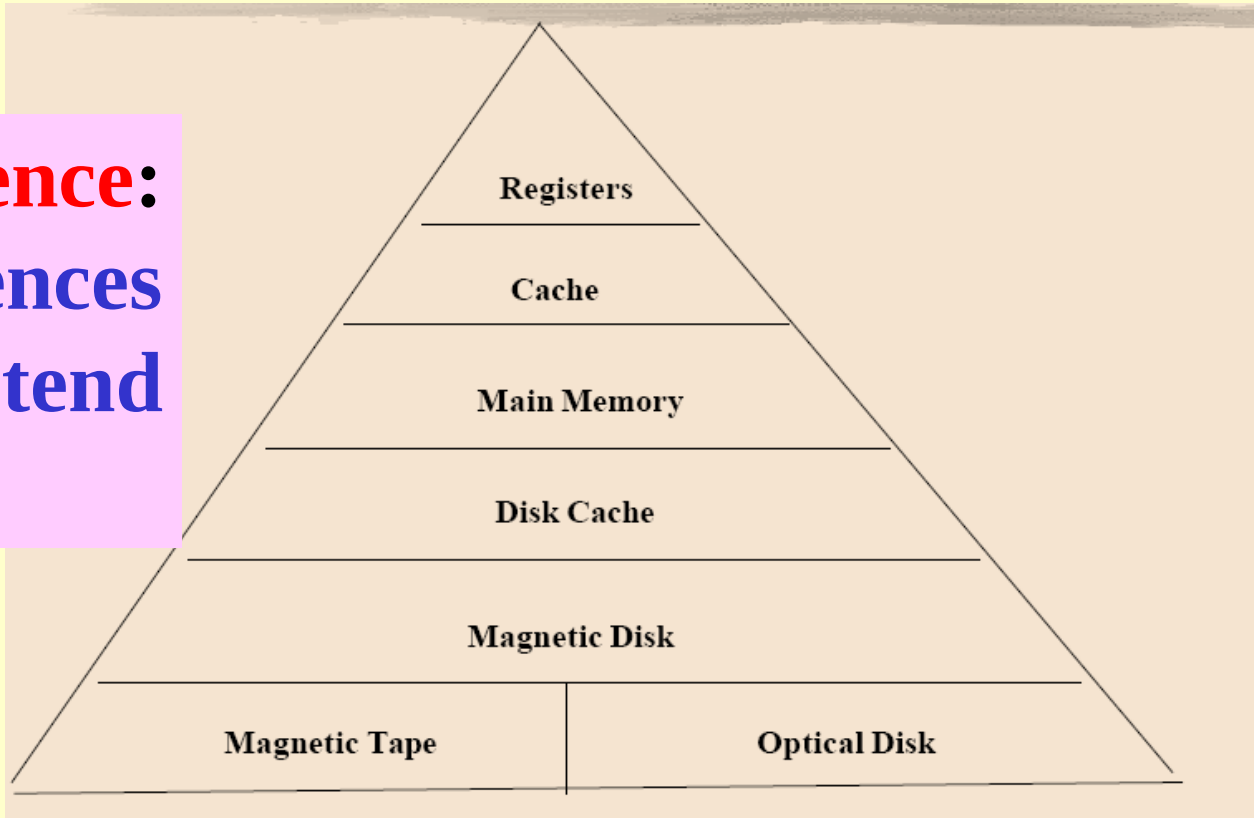
Greater capacity, smaller cost per bit.

Greater capacity, slower access speed.

Going Down the Hierarchy

- Decreasing cost per bit — Increasing capacity
- Increasing access time
- Decreasing frequency of access of the memory by the processor

locality of reference: memory references by the processor tend to cluster.



- **Main Memory (MM) is the principal internal memory system of the computer.**

- **Each location in main memory has a unique address.**

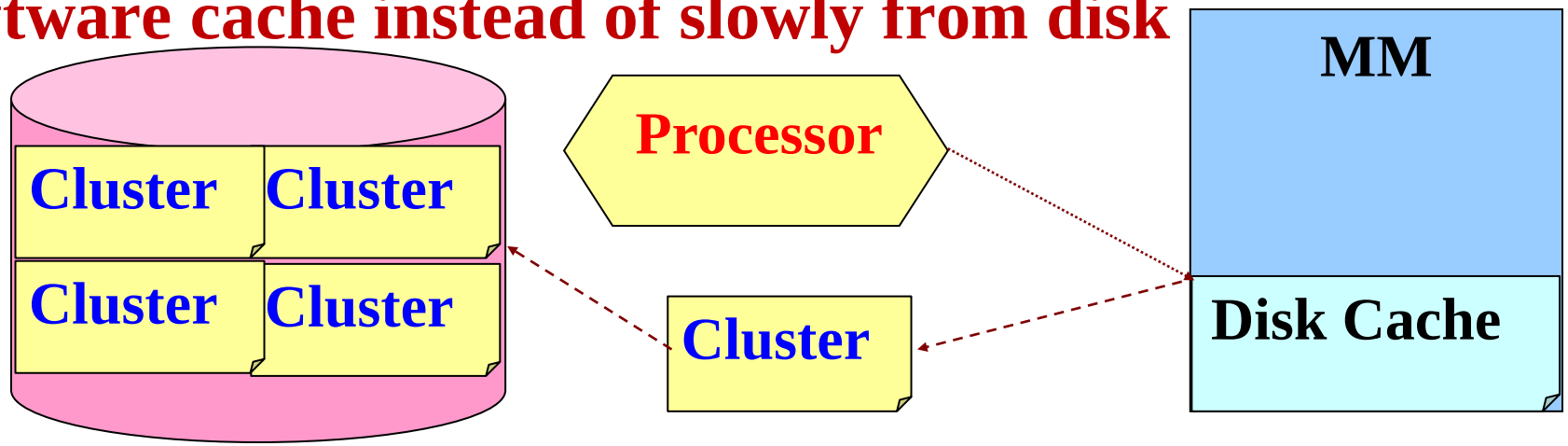
- **Semiconductor memory comes in a variety of types, which differ in speed and cost.**

Disk Cache

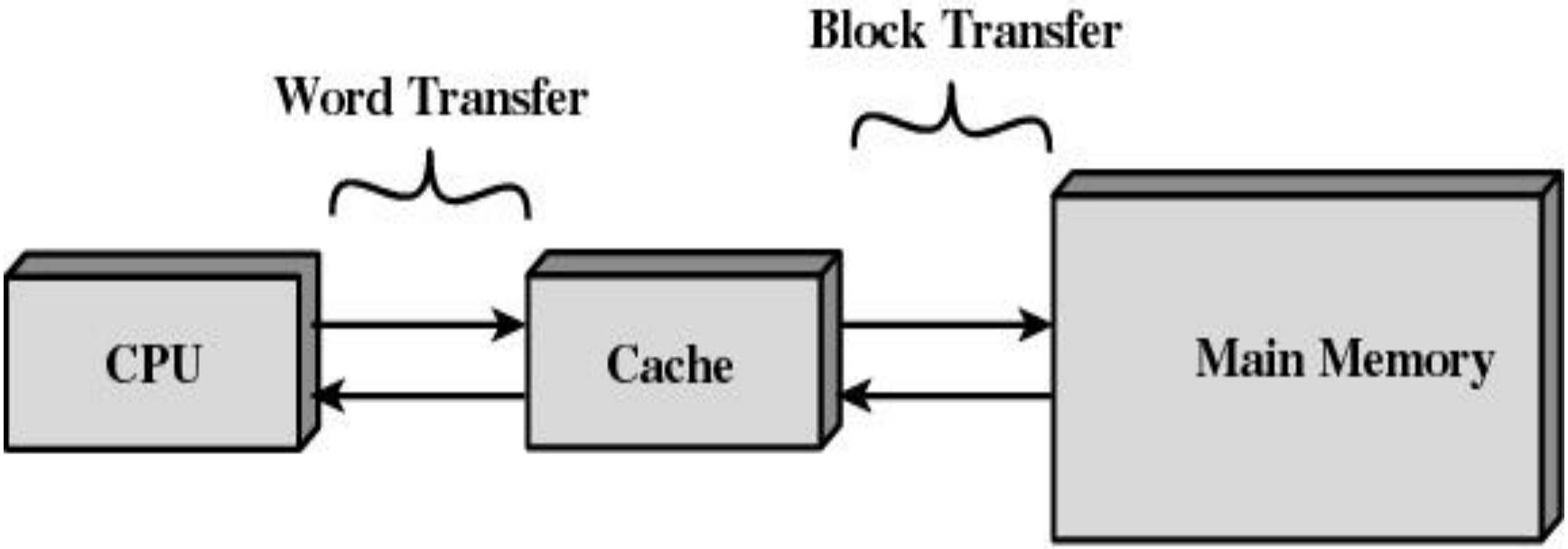
- A portion of main memory used as a buffer to temporarily hold data that are to be read out to disk.

- Disk cache improves performance in two ways:
- 1- Disk writes are clustered: Instead of many small transfers of data, we have a few large transfers of data. This improves disk performance and minimizes processor involvement.

2- Some data destined for write-out may be referenced again. The data are retrieved rapidly from the software cache instead of slowly from disk



1.6 CACHE MEMORY



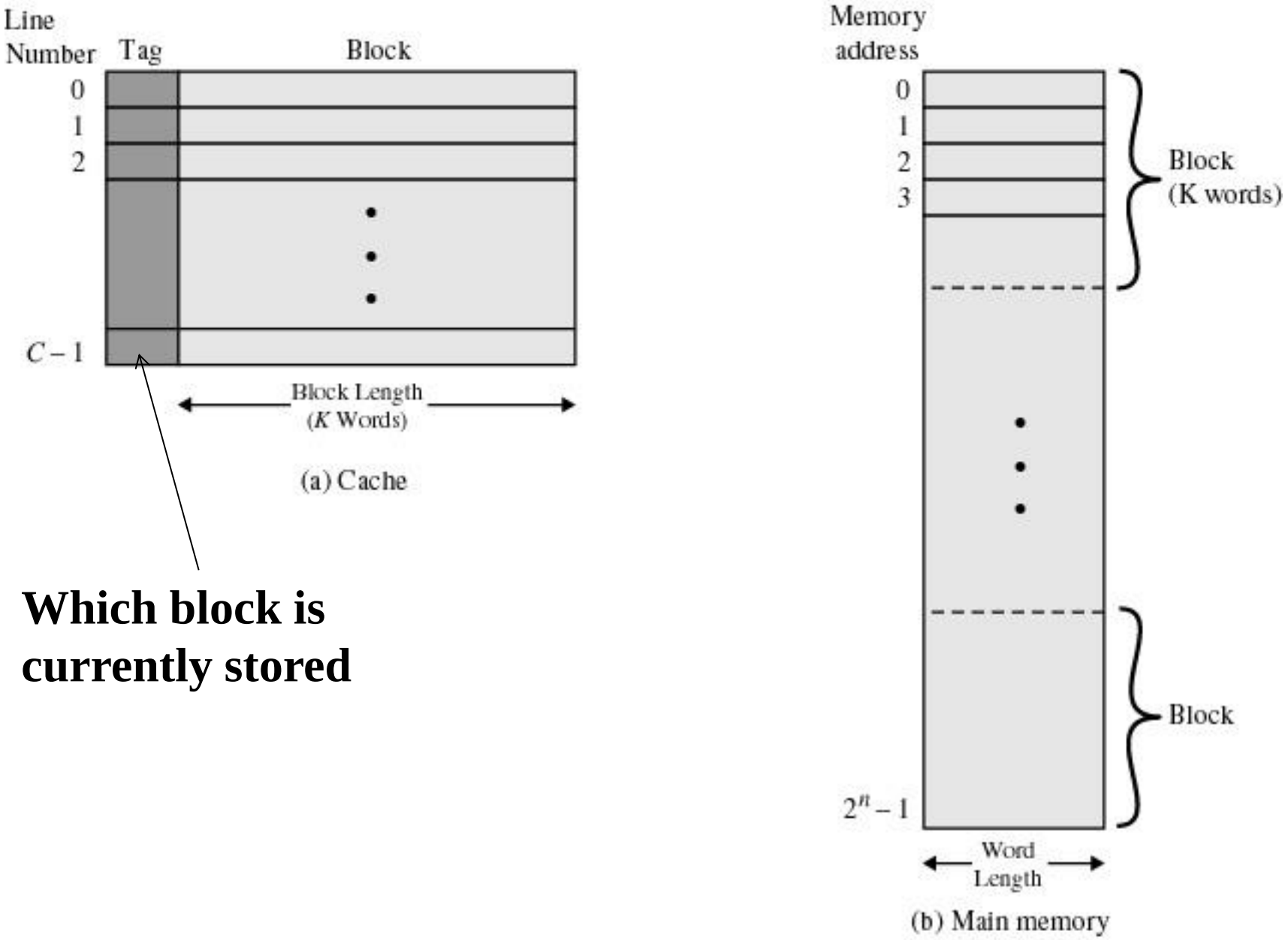
The cache is a device for staging the movement of data between memory and processor registers to improve performance.

- **Invisible to operating system**
- **On all instruction cycles, the processor accesses memory at least once, to fetch the instruction, and one or more to fetch operands and store results.**
- **Processor speed is faster than memory speed, The rate at which the processor can execute instructions is limited by the memory cycle time.**
- **Should we build the main memory using the same technology as that of the processor registers?**

  Exploiting the principle of locality by providing a small fast memory between the processor and main memory, the cache.

Cache Memory

- **Contains a portion of main memory.**
- **Processor first checks cache.**
- **If not found in cache, the block of memory containing the needed information is moved to the cache**
- **Due to the locality reference, many of the near-future memory references will be to other bytes in the block.**



Which block is currently stored

Figure 1.17 Cache/Main-Memory Structure

Cache Design

- **Cache size**
  - **small caches have a significant impact on performance**
- **Block size**
  - **the unit of data exchanged between cache and main memory**
  - **hit means the information was found in the cache**
  - **larger block size more hits until probability of using newly fetched data becomes less than the probability of reusing data that has been moved out of cache**

Cache Design

- **Mapping function**
  - **determines which cache location the block will occupy**
- **Replacement algorithm**
  - **determines which block to replace**
  - **Least-Recently-Used (LRU) algorithm**

Cache Design

- **Write policy**

  When the memory write operation takes place

  1- Can occur every time block is updated

  2- Can occur only when block is replaced

  - **Advantage: Minimizes memory operations**
  - **Disadvantage :Leaves memory in an obsolete state**

1.7 I/O COMMUNICATION TECHNIQUES

Three techniques are possible for I/O operations:

-Programmed I/O

-Interrupt-driven I/O

-Direct memory access I/O

1- Programmed I/O

When an I/O instruction occurs:

1- the processor issues a command to the appropriate I/O module.

2- the I/O module:

- **performs the requested action**
- **sets the appropriate bits in the I/O status register**
- **takes no action to alert the processor**

(does not interrupt the processor)

3- the processor must determine when the I/O instruction is completed

periodically checks the status of the I/O module

The processor is responsible for:

-extracting data from memory for output

-Storing data in main memory for input

The instruction set (IS) includes I/O instructions in the following categories:

-control: used to activate an external device and tell it what to do.

A magnetic tape unit may be instructed to rewind or to move forward one record.

-Status: used to test various status conditions associated with an I/O module and its peripherals.

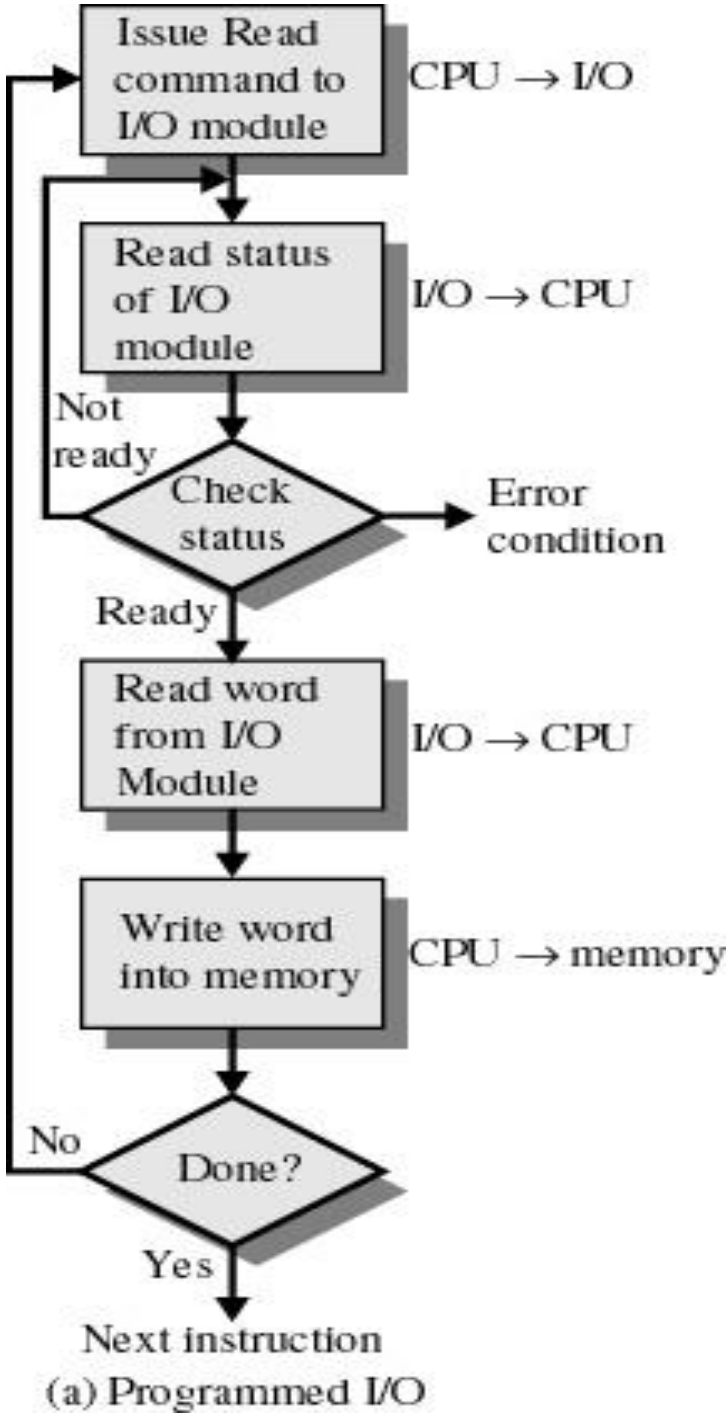
-Transfer: used to read and /or write data between processor registers and external devices.

1- Programmed I/O

- **I/O module performs the action, not the processor**
- **Sets appropriate bits in the I/O status register**
- **No interrupts occur**
- **Processor checks status until operation is complete**

The main disadvantage of this technique:

It is a time consuming process that keeps the processor busy needlessly.



(a) Programmed I/O

2- Interrupt-Driven I/O

1- The processor issue an I/O command to a module.

2- The processor execute other useful work.

3- The I/O module interrupts the processor when it is ready to exchange data with the processor.

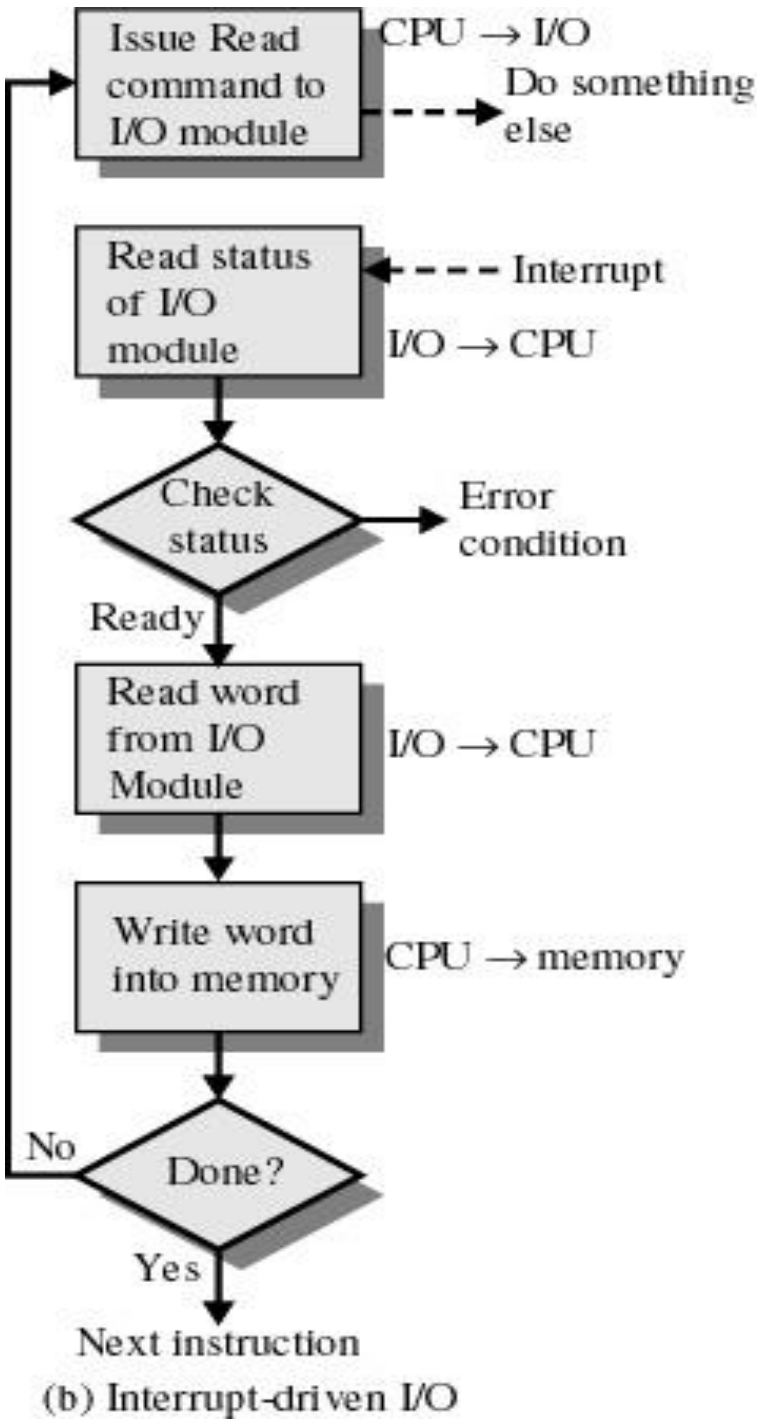
4- The processor executes the data transfer, and then resumes its former processing.

2-Interrupt-Driven I/O

- **Processor is interrupted when I/O module ready to exchange data**
- **Processor is free to do other work**
- **No needless waiting**

The main disadvantage of this technique:

Consumes a lot of processor time because every word of data goes from memory to I/O module or from I/O module to memory must passes through the processor



(b) Interrupt-driven I/O

3- Direct Memory Access

programmed I/O and Interrupt-Driven I/O Both of them suffer from two drawbacks:

1- the I/O transfer rate is limited by the speed with which can test and service a device.

2- the processor is tied up in managing an I/O transfer.

3- Direct Memory Access

1- The processor issues a command to the DMA module, sending the following information:

-Whether a read or write is requested.

-The address of the I/O device involved.

-The starting location in memory to read or write.

-The number of words to be read or written.

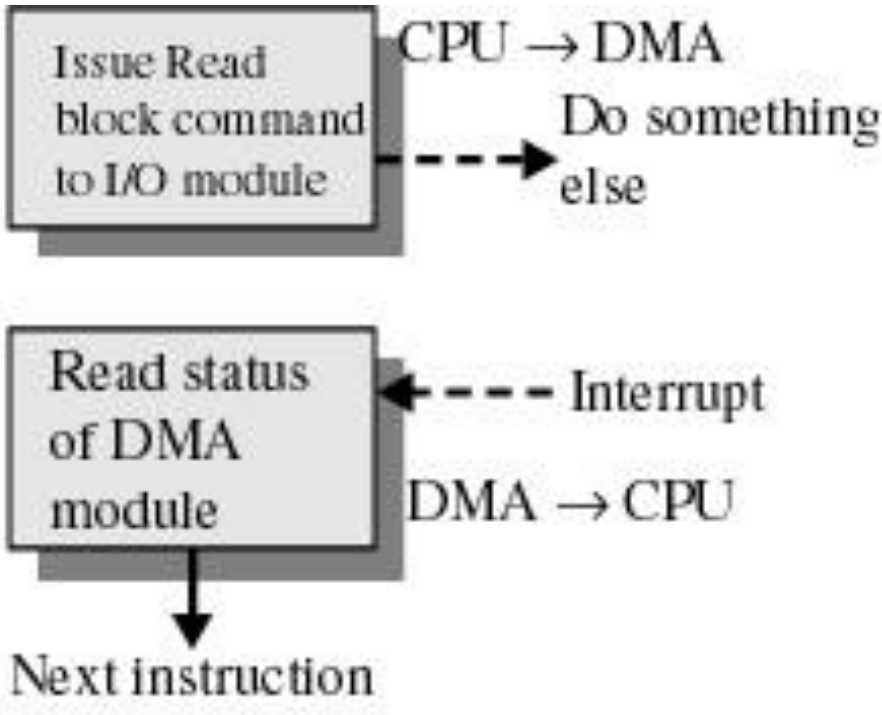
2- The processor execute other useful work.

3- The DMA module transfer the entire block of data, one word at a time, directly to or from memory (without going through the processor)

4- The DMA module, when the transfer is complete, sends an interrupt signal to the processor.

3- Direct Memory Access

- **Transfers a block of data directly to or from memory**
- **An interrupt is sent when the task is complete**
- **The processor is only involved at the beginning and end of the transfer**



(c) Direct memory access